



Claude Code 技术架构拆解

作者：JZNext

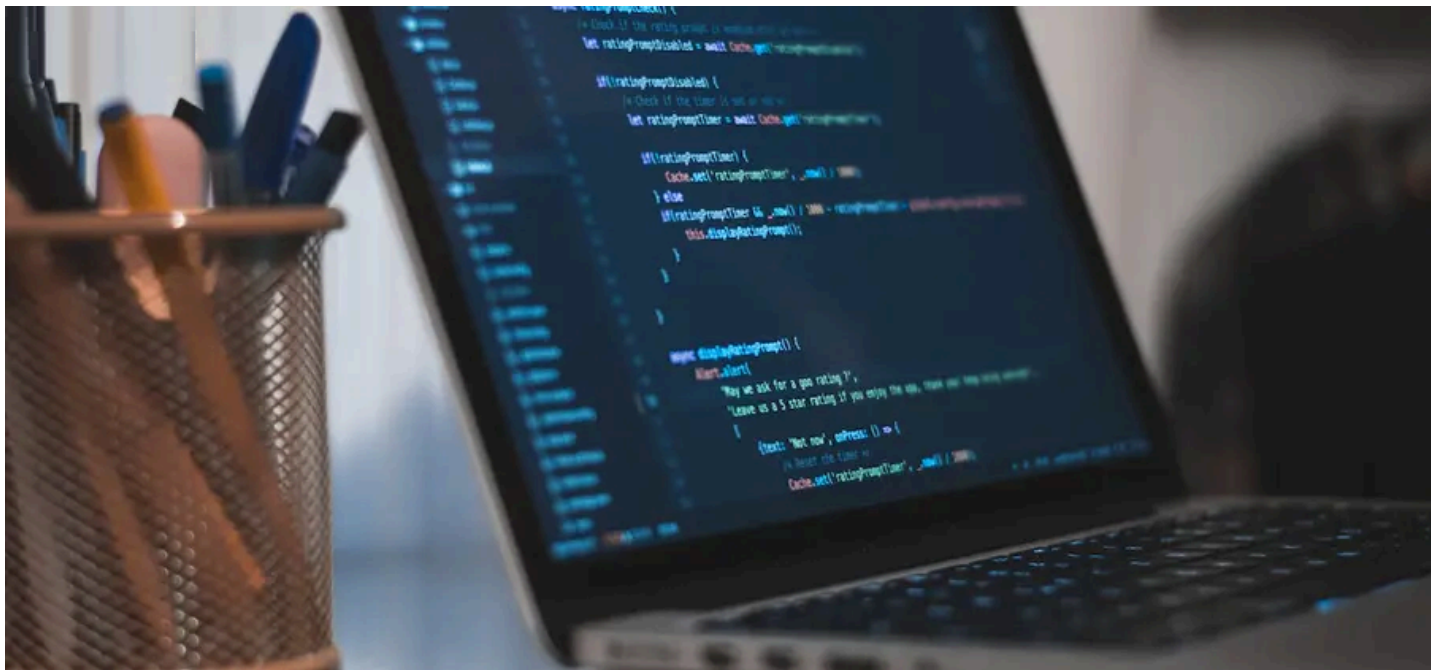
2026-4-8 384 次浏览 6 次

编辑推荐:

本文将深度拆解其核心模块设计，揭示如何让 AI 安全高效地接管开发工作流，希望对你的学习有帮助。

本文来自于人人都是产品经理，由火龙果软件Alice编辑，推荐。

Claude Code 正在重新定义 AI 编程助手的边界——它不是简单的代码补全工具，而是直接在终端里运行的 AI 开发操作系统。这款由 Anthropic 开源的 CLI 工具能自主读写文件、执行命令、操作 Git，其分层架构设计、权限模型和多 Agent 协作机制，为 AI 产品经理提供了教科书级的架构范本。本文将深度拆解其核心模块设计，揭示如何让 AI 安全高效地接管开发工作流。



Claude Code 是 Anthropic 开源的 AI 编程助手，产品形态是一个跑在终端里的 CLI 工具。开发者在命令行里用自然语言告诉它想做什么，它就能直接读写文件、执行 Shell 命令、操作 Git、调用外部服务，把一个编程任务从头到尾干完。这不是又一个“代码补全插件”。它的野心更大——做一个终端原生的 AI 开发操作系统。

作为 AI 产品经理，理解 Claude Code 的架构设计有三层价值：

第一，它是目前工程成熟度最高的开源 AI Agent 实现之一，1902 个 TypeScript 文件，每一个架构决策都经过了真实产品场景的打磨；

第二，它的分层设计、权限模型、多 Agent 协调机制，对任何 AI 产品的架构规划都有直接参考价值；

第三，读懂它，你和工程团队讨论技术方案时就能说到点子上。

产品定位与核心能力

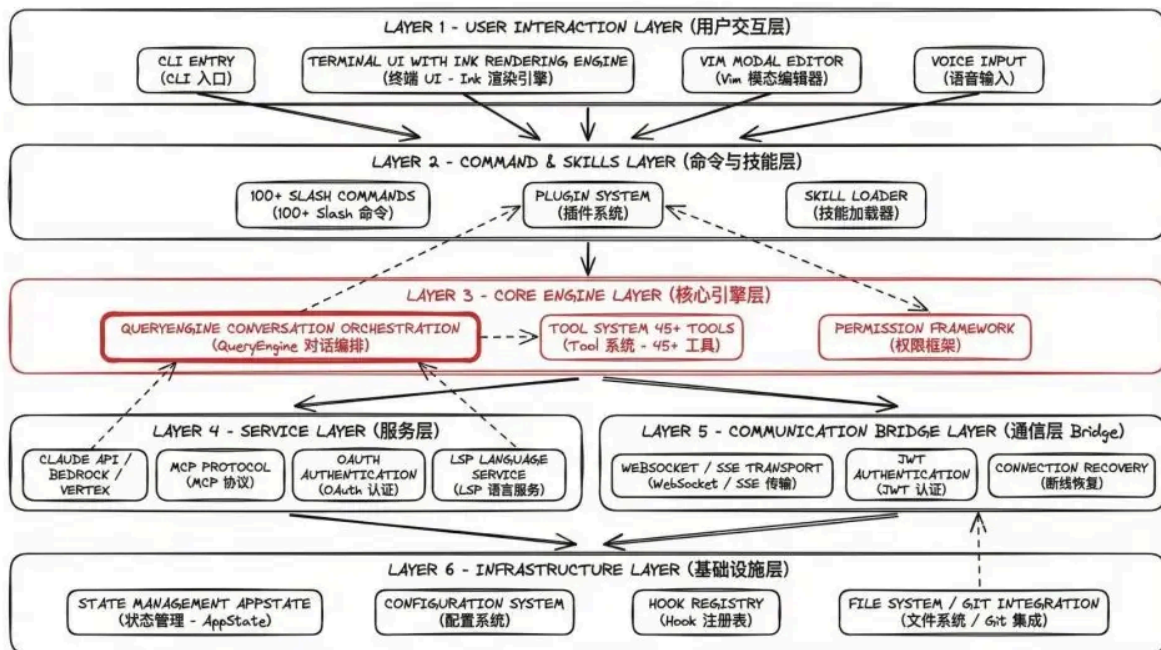
Claude Code 的核心能力可以拆成五个具体的东西：

1. 终端交互 —— 用户在命令行输入自然语言指令，Claude Code 理解意图后调用对应工具执行。不是生成代码片段让你复制粘贴，而是直接操作你的项目文件和开发环境。
2. 工具调用 —— 内置 45+ 工具，覆盖文件读写 (FileRead、FileEdit、Write)、Shell 命令执行 (BashTool)、代码搜索 (Glob、Grep)、Git 操作等。每个工具都有独立的权限控制和进度追踪。
3. 多 Agent 协作 —— 支持创建并行子代理，一个主 Agent 拆分任务给多个子 Agent 同时干活。团队模式下多个 Agent 可以通过消息协议互相通信。
4. 远程会话 —— 通过 Bridge 系统，用户在 claude.ai 网页端可以远程控制本地的 CLI 实例。在公司电脑上跑着 CLI，回家用浏览器继续操作。
5. 可扩展性 —— 100+ 内置 Slash 命令、MCP 协议集成、插件/技能系统。第三方开发者可以通过标准协议接入自己的工具和服务。

和 GitHub Copilot、Cursor 等竞品相比，Claude Code 的本质差异在于：它不依赖 IDE，不做代码补全，而是把整个开发环境当作操作对象，通过 Agent Loop 自主完成任务链路。这是“辅助工具”和“自主代理”的根本区别。

整体架构设计

Claude Code 的架构分成六层，从上到下依次是用户交互层、命令与技能层、核心引擎层、服务层、通信层和基础设施层。



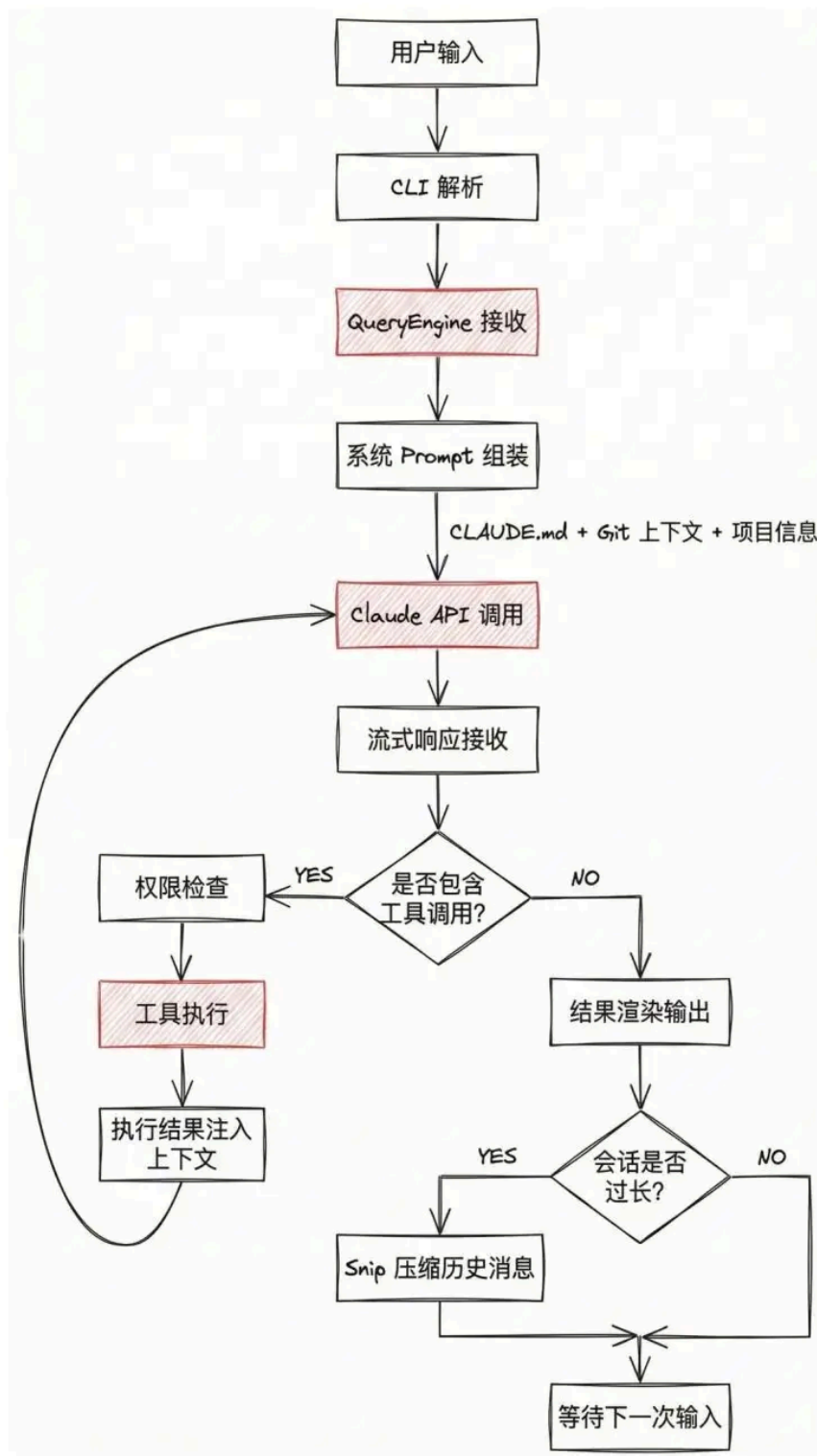
逐层看每一层的职责和存在理由：

1. 用户交互层 负责接收用户输入和展示结果。这一层的关键决策是自研 Ink 终端渲染引擎而不是用现成的 blessed、ink 等库。原因很实际：AI Agent 的输出是流式的、长文本的、需要实时更新的，现有终端 UI 库在这个场景下的性能和灵活性都不够。自研引擎保证了 16ms 级别的渲染刷新和流畅的滚动体验。
2. 命令与技能层 是用户意图到系统行为的翻译层。100+ Slash 命令覆盖了开发者日常工作流：/commit、/diff、/pr_comments 处理 Git 工作流，/tasks、/agents 管理多 Agent 任务，/mcp、/skills 接入外部能力。这一层的产品价值在于：降低用户的学习门槛，把复杂的 Agent 能力包装成熟悉的命令行交互模式。
3. 核心引擎层 是整个系统的大脑。QueryEngine 编排对话流程，Tool 系统定义能力边界，权限框架控制安全底线。这三个模块的耦合度很低——QueryEngine 不关心具体有哪些工具，Tool 系统不关心对话上下文怎么管理，权限框架独立于两者运行。这种解耦设计让每个模块可以独立迭代。
4. 服务层 对接外部依赖。Claude API 支持 Anthropic 直连、AWS Bedrock、Azure Foundry、Vertex AI 四种接入方式，自动重试加指数退避。MCP 协议支持 stdio、SSE、WebSocket、HTTP 四种传输方式。这一层抽象掉了外部服务的差异，上层不需要关心底层用的是哪家云厂商。
5. 通信层 解决一个具体的产品场景：用户想在网页端远程操作本地 CLI。Bridge 系统通过 WebSocket/SSE 建立双向通道，JWT 做认证，崩溃后能自动恢复会话。
6. 基础设施层 提供全局共享的底层能力：Deep Immutable 的状态树保证数据一致性，配置系统支持多级优先级覆盖 (CLI 参数 > 本地设置 > 项目设置 > 用户设置)，Hook 注册表让扩展点可以被外部代码挂载。

核心模块深度拆解

QueryEngine: 13000 行代码的对话编排中枢

QueryEngine 是 Claude Code 最核心的模块，每个对话实例创建一个 QueryEngine 实例。它的职责用一句话概括：把用户的输入变成一次完整的 AI 交互，中间处理好上下文组装、工具调度、Token 管理、异常恢复等所有脏活。

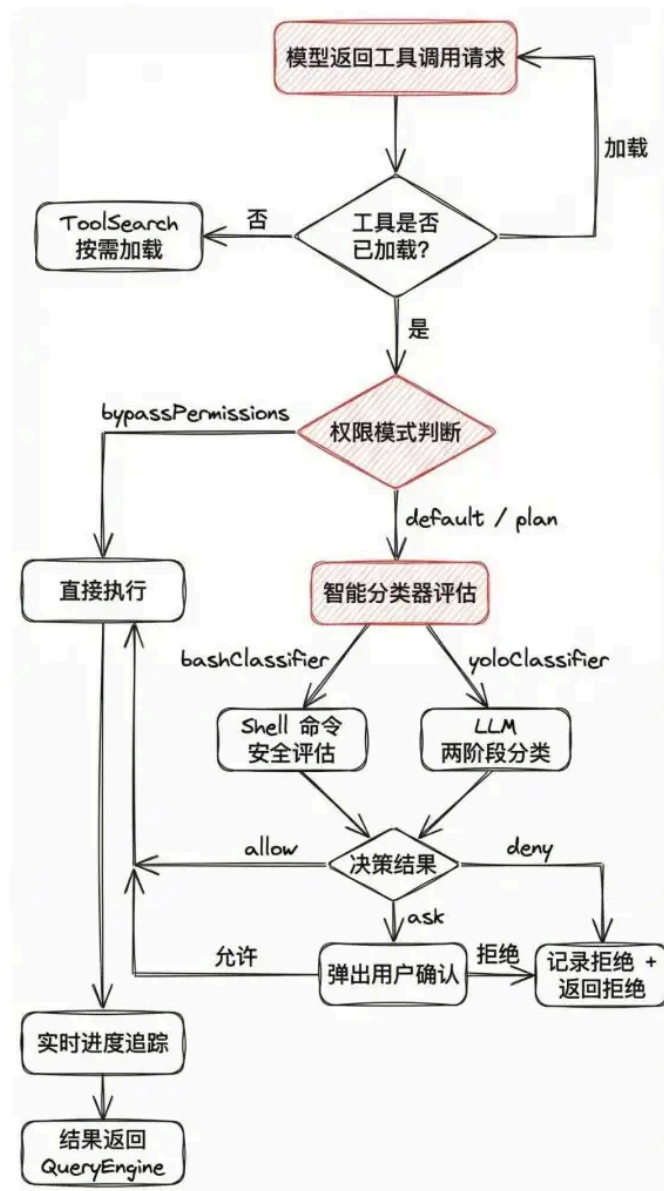


几个关键的设计决策值得拆开讲：

- 系统 Prompt 组装 不是一段固定文本，而是动态拼接的。QueryEngine 会读取项目根目录的 CLAUDE.md 文件（类似项目级的记忆文件），获取当前 Git 分支和最近提交信息，收集项目结构和依赖信息，然后把这些内容拼成完整的系统 Prompt。这意味着同一个用户在不同项目里使用 Claude Code，得到的是不同的“AI 助手人格”。
- Snip 压缩 解决的是一个实际的工程问题：Claude API 有上下文窗口限制，长对话会超出 Token 上限。Snip 机制在对话变长时自动截断历史消息，但不是简单地删掉旧消息，而是保留语义连续性——压缩后的上下文仍然能让模型理解之前在干什么。这个机制对产品体验的影响很大：用户可以在一个会话里连续工作几小时，不会因为上下文溢出而丢失前面的工作进度。
- 推测执行 (Speculation) 是一个提升响应速度的设计。在用户还没输入下一条指令时，QueryEngine 会根据当前上下文预判用户可能的下一步操作，提前生成建议。这类似于搜索引擎的“搜索建议”，但发生在 Agent 的任务执行链路上。

Tool 系统：45 个工具背后的设计哲学

Claude Code 的工具不是随便堆的。每个工具都是一等公民，拥有完整的类型定义、权限声明、渲染逻辑和进度追踪机制。



工具分成六类：

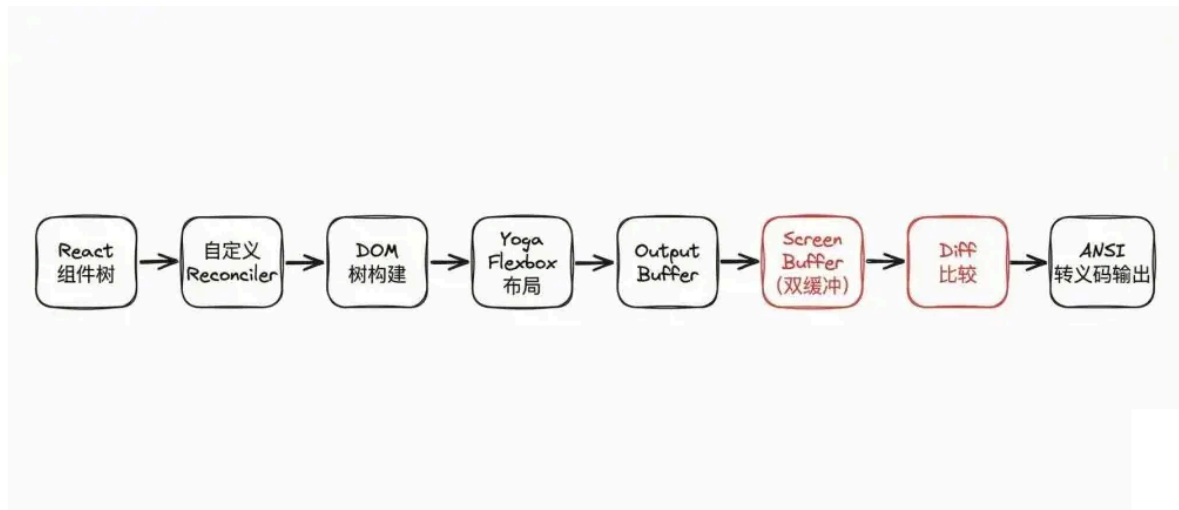
工具类别	代表工具	产品场景
文件操作	FileRead、FileEdit、Write、Glob、Grep	读写代码文件、搜索项目内容
执行环境	BashTool	执行 Shell 命令，捕获标准输出和错误
代理系统	AgentTool	创建并行子代理执行子任务
用户交互	AskUserQuestion	遇到歧义时主动向用户提问
工作流	EnterPlanMode、EnterWorktree	切换到计划模式或 Git 工作树
外部集成	MCPTool、SkillTool、LSPTool	调用 MCP 服务器、技能插件、语言服务

延迟加载 是一个影响启动速度的关键设计。45 个工具不是启动时全部加载，而是通过 ToolSearch 按需加载。核心工具（FileRead、BashTool 等）常驻内存，非核心工具（LSPTool、SkillTool 等）首次使用时才加载。实际效果：CLI 启动时间从秒级降到毫秒级。

MCP（Model Context Protocol）集成 让 Claude Code 的工具体系变成开放的。任何实现了 MCP 协议的外部服务器，都可以把自己的工具暴露给 Claude Code 使用。这意味着第三方开发者不需要修改 Claude Code 的源码，就能给它增加新能力。

Ink 终端渲染引擎：为什么要自己造轮子

Claude Code 的终端 UI 不是用 React Ink 库简单包装的，而是从 Reconciler 层开始重写的自研渲染引擎，共 80+ 文件。



为什么不用现成方案？三个具体原因：

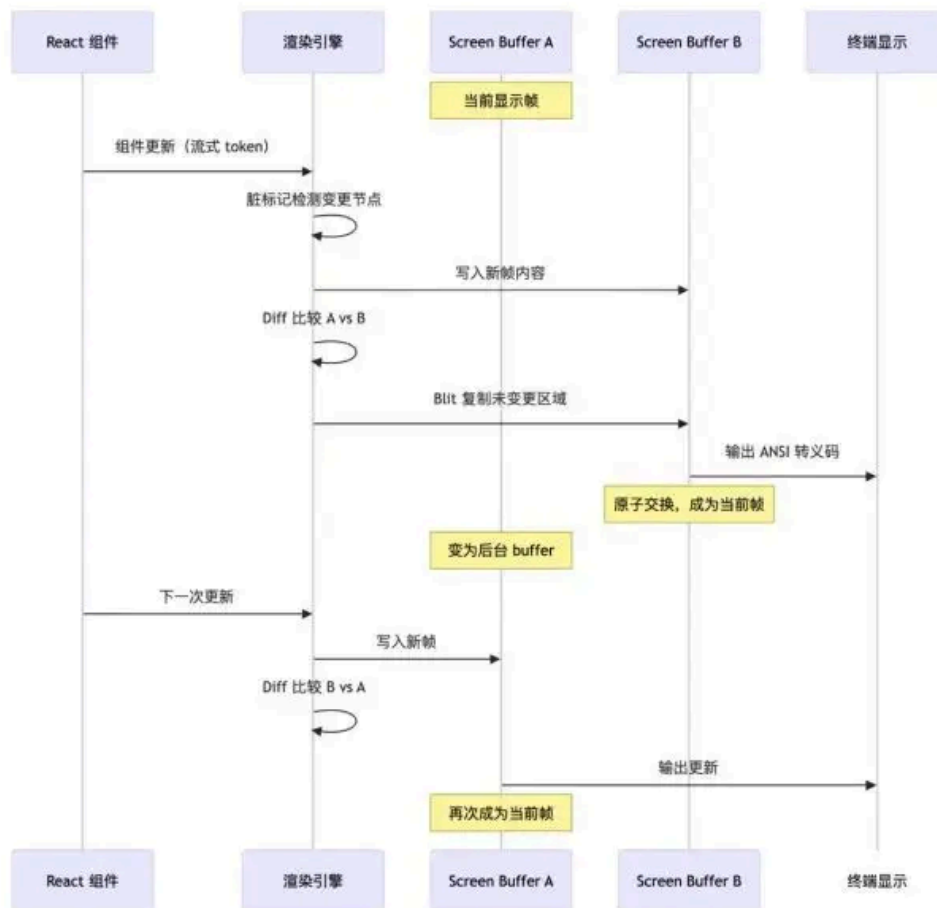
第一，流式输出的性能问题。AI 模型的响应是 token 级别的流式输出，每秒可能触发几十次 UI 更新。通用终端 UI 库在这种频率下会出现明显的闪烁和卡顿。Claude Code 的双缓冲渲染方案——两个 Screen Buffer 交替写入和显示，前后帧原子交换——把闪烁问题彻底消除了。

第二，长文本滚动的体验问题。AI 生成的代码可能有几百行，用户需要在终端里顺畅滚动查看。自研引擎使用 DECSTBM 硬件滚动区域指令，把滚动操作交给终端模拟器硬件处理，配合 ScrollBox 的视口裁剪（只渲染可见行），滚动体验和原生终端一样流畅。

第三，布局复杂度。Claude Code 的 UI 包含多栏布局、进度条、代码高亮、折叠面板等复杂元素。Yoga Flexbox 引擎提供完整的弹性布局支持，让终端 UI 的布局能力接近 Web 页面。

性能优化的细节很多：脏标记传播避免无效重绘，Blit 快速路径直接复制未变更区域，字符池和样式池做内存复用，渲染节流控制在 16ms 一帧。这些优化加在一起，让 Claude Code 在低配终端上也能流畅运行。

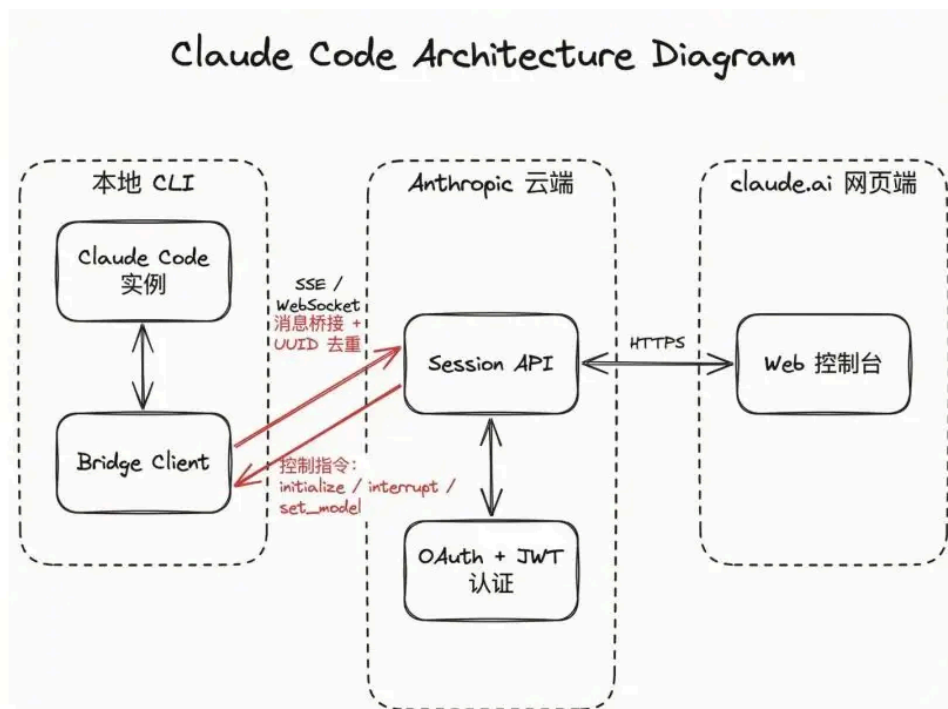
双缓冲渲染的工作原理：



这个机制的关键价值：AI 流式输出每秒可能触发 30-50 次 UI 更新，双缓冲保证了前后帧的原子交换，用户看到的画面永远是完整的一帧，不会出现“半帧闪烁”。

Bridge 远程会话：解决“我不在电脑前”的问题

Bridge 系统的产品场景很具体：开发者在办公室的电脑上跑着 Claude Code CLI，回到家想在 claude.ai 网页端继续操作，或者想在手机浏览器上查看任务进度。



Bridge 提供两种架构模式：

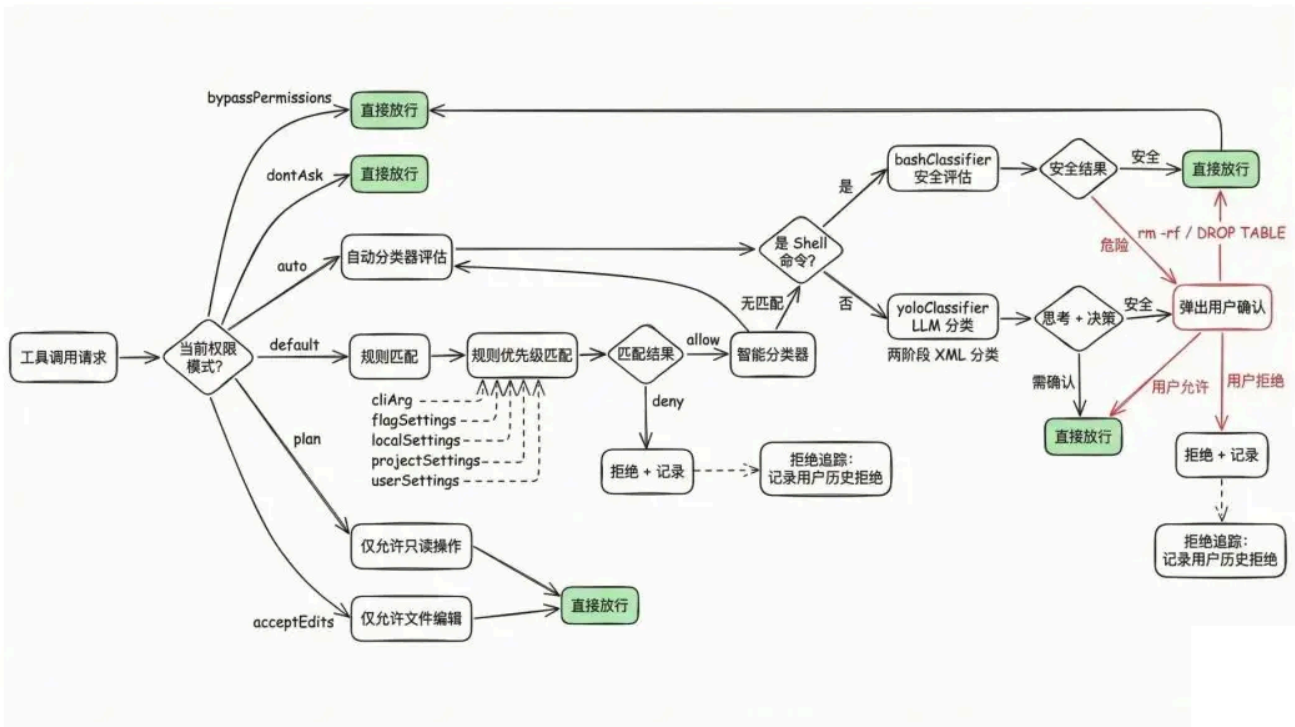
Environment-based 模式 适合需要管理多个会话的场景。本地 CLI 注册一个“环境”，云端分配 environment_id，然后在这个环境下创建多个会话。这种模式的好处是会话可以独立管理，一个环境里可以同时跑多个任务。

Env-less 模式 适合简单的交互式场景。跳过环境注册，直接通过 OAuth 拿到 token，建立单个会话。启动更快，但只支持一个活跃会话。

崩溃恢复是这个系统的硬需求。如果网络断开或进程意外退出，Bridge 会把 environment_id 和 session_id 持久化到 bridgePointer.json 文件。重启后读取这个文件，重新建立连接，通过序列号跟踪补发丢失的消息。UUID 环形缓冲做去重，防止恢复过程中消息重复投递。

权限系统：安全不是可选项

Claude Code 让 AI 直接操作文件系统和执行 Shell 命令，安全是生死问题。权限系统不是一个简单的“允许/拒绝”开关，而是一个多层决策模型。



六种权限模式对应六种不同的产品场景：

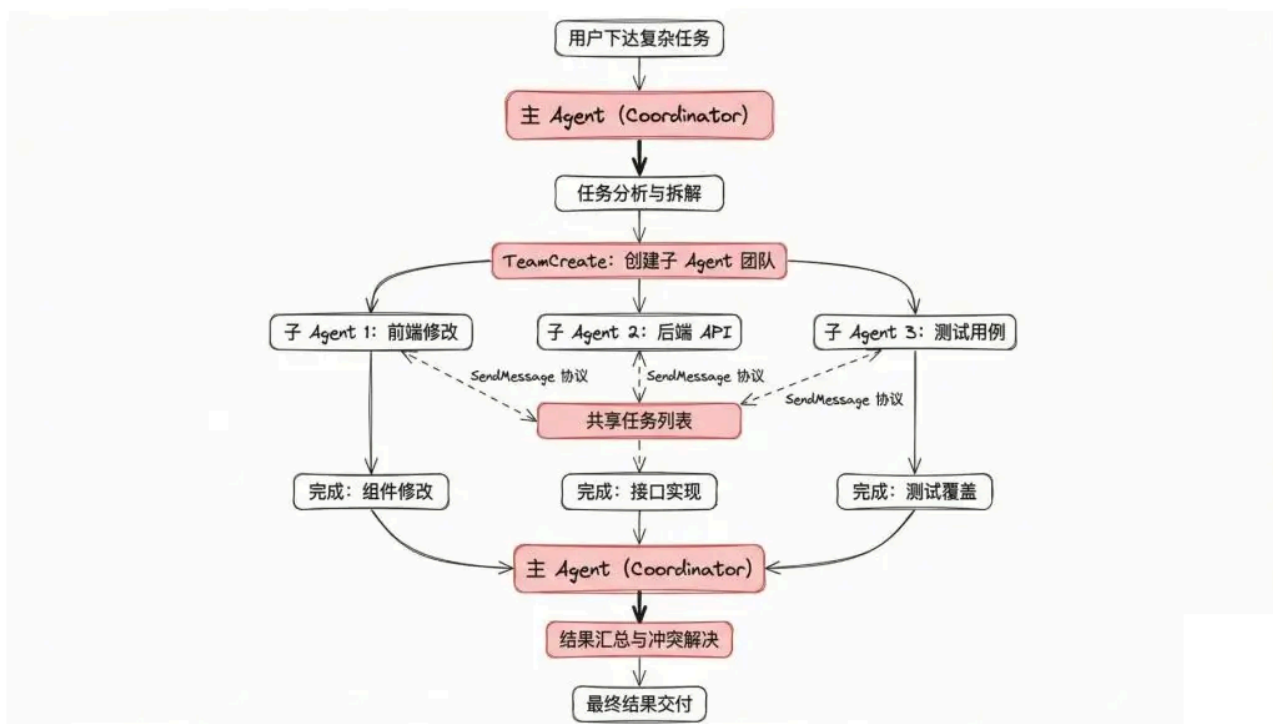
1. default：标准模式，危险操作需要用户确认
2. plan：计划模式，只允许只读操作，AI 只能“看”不能“动”
3. acceptEdits：允许文件编辑但不允许执行命令
4. bypassPermissions：完全信任模式，适合 CI/CD 自动化场景
5. dontAsk：不弹确认框，但仍然记录
6. auto：由分类器自动判断

bashClassifier 做的事情很直接：检查 Shell 命令是否包含 rm -rf、DROP TABLE、chmod 777 等已知危险模式。yoloClassifier 更复杂，它调用 LLM 做两阶段 XML 分类——先生成思考过程，再给出决策结果。

拒绝追踪 是一个容易被忽略但很重要的设计。系统会记录用户历史上拒绝过的操作类型，下次遇到类似操作时自动提高警戒级别。这避免了一个常见问题：用户拒绝了一个危险操作，但 AI 换个措辞又来一遍。

多 Agent 协作：一个 Agent 不够用的时候

单个 Agent 处理复杂任务有两个瓶颈：上下文窗口有限，串行执行太慢。Claude Code 的多 Agent 架构直接解决这两个问题。



主 Agent 作为协调器 (Coordinator)，负责任务拆解和结果汇总。子 Agent 通过 TeamCreate 协议创建，通过 SendMessage 协议互相通信，通过共享任务列表同步状态。

每个子 Agent 有独立的上下文窗口，这意味着一个 200k token 上下文的任务可以拆成三个 70k token 的子任务并行处理。任务总量没变，但每个 Agent 的上下文压力减小了，回答质量更高。

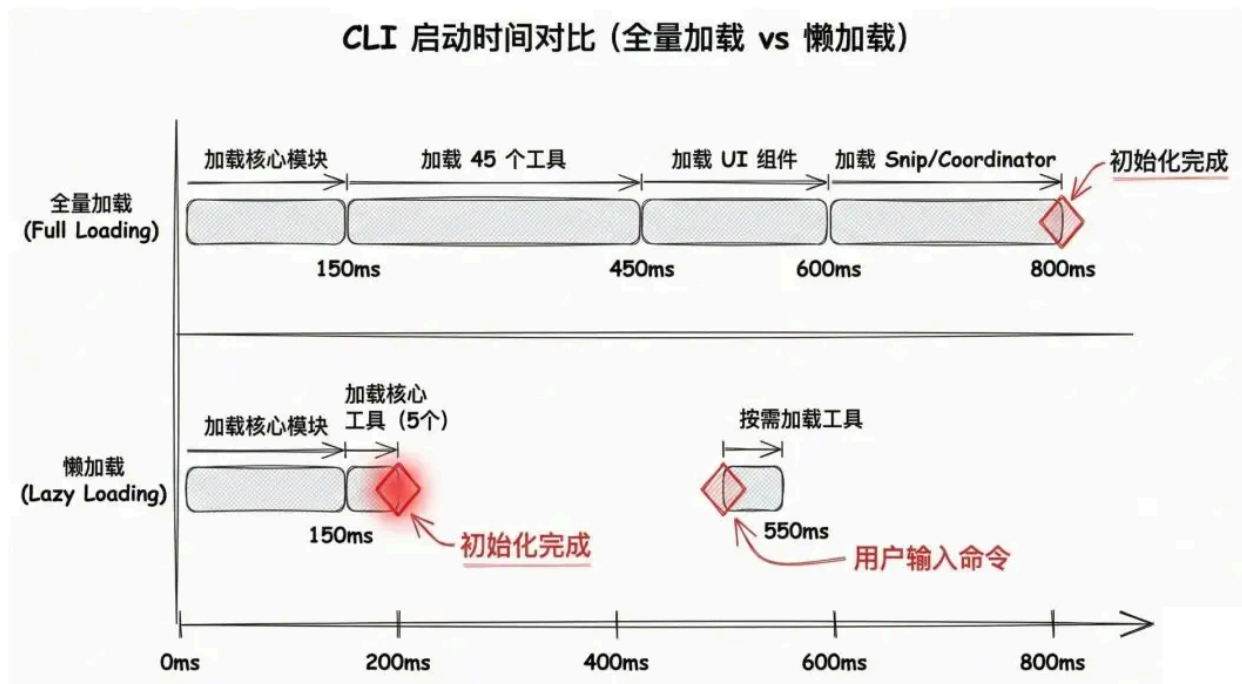
并行执行的速度优势也很明显。三个子任务如果串行处理需要 3 分钟，并行处理只需要 1 分钟多。

架构设计亮点

拉远视角看整个架构，有几个设计决策特别值得关注。

懒加载贯穿全局。不只是工具系统做延迟加载，Snip 压缩模块、Coordinator 协调器、非核心 UI 组件都通过 feature() 函数按需加载。这种设计的产品影响是：CLI 首次启动时只加载必要模块，用户输入第一条命令的等待时间控制在毫秒级。对于一个命令行工具来说，启动速度直接决定用户是否愿意每天使用。

懒加载的启动时间对比：



启动时间从 800ms 降到 200ms，用户感知的等待时间减少了 75%。非核心工具在首次使用时才加载，对后续操作几乎无感知。

- 双缓冲渲染的工程实现 不只是“两个 buffer 换来换去”这么简单。脏标记传播机制追踪哪些节点发生了变化，Blit 快速路径让未变更区域直接从旧 buffer 复制到新 buffer，字符池和样式池做内存复用避免 GC 压力，Yoga 布局引擎的单槽缓存在布局参数未变时直接返回上次结果。这些优化叠加在一起，保证了 AI 流式输出时终端不卡顿。
- 传输抽象的可扩展性 体现在 Bridge 系统的 v1/v2 传输层设计上。v1 用 WebSocket+POST，v2 用 SSE+CCR (Claude Communication Record)，应用层代码完全不感知底层用的是哪种传输协议。当 Anthropic 需要从 WebSocket 迁移到 SSE 时，只改传输层代码，上面所有的会话管理、消息路由、崩溃恢复逻辑一行不动。
- 推测执行对用户的影响 看起来大。AI Agent 的一次任务执行通常包含多个来回：调用工具→看结果→决定下一步→再调用工具。每一次来回都有网络延迟和模型推理延迟。推测执行在用户还在看结果时，就预判下一步可能的操作并提前准备好。用户看完结果点确认时，下一步操作几乎瞬间完成。
- 插件化扩展的三位一体设计——技能 (Skills)、钩子 (Hooks)、MCP 服务器——覆盖了三种不同的扩展场景。技能是预定义的工作流模板（比如“前端组件开发技能”），钩子是在特定事件点执行自定义逻辑（比如“每次提交前跑 lint”），MCP 服务器是完整的外部工具接入。三种机制各有适用场景，不互相替代。

产品经理可以借鉴什么

Claude Code 的架构设计有几个思路，对做 AI 产品的产品经理有直接参考价值。

分层架构不是技术偏好，而是产品迭代速度的保障。

Claude Code 的六层架构让每一层可以独立迭代。要换大模型供应商？改服务层就行。要加新的交互方式（比如语音）？改用户交互层就行。要接入新的外部工具？改 MCP 集成就行。产品经理规划功能迭代时，如果架构分层合理，每个新功能的影响范围是可预期的，排期就不会经常翻车。

“权限即代码”意味着安全不是后补的。

很多 AI 产品的安全机制是在出了安全事故后才加上的，导致权限逻辑散落在代码各处，难以维护也容易遗漏。Claude Code 的做法是每个工具声明自带权限元数据，运行时由统一的权限框架校验。产品经理在设计 AI 产品时，应该在 PRD 阶段就把权限模型写清楚，而不是等开发完了再补。

性能优化是用户体验的一部分。

双缓冲渲染、懒加载、推测执行——这些技术手段的最终目的都是让用户觉得“这个工具很快”。在 AI 产品中，模型推理延迟是不可避免的，但通过架构层面的优化（流式输出、预加载、并行处理），可以把用户感知到的等待时间压缩到可接受的范围。

可扩展性要在第一天就设计好。

Claude Code 的 MCP 协议集成、技能系统、Hook 机制，都不是后加的，而是从架构设计初期就预留了扩展点。一个 AI 产品如果想让第三方开发者参与进来，扩展接口必须是架构的一等公民，不是一个“也可以扩展”的附属功能。

多 Agent 不是噱头，是解决具体性能瓶颈的方案。

不要因为“多 Agent”听起来很酷就在产品里加这个功能。Claude Code 引入多 Agent 是因为单 Agent 的上下文窗口和串行执行速度确实限制了复杂任务的处理能力。做产品决策时，先确认单 Agent 的能力天花板在哪里，再决定是否引入多 Agent 架构。

写在最后

Claude Code 的 1902 个 TypeScript 文件背后，是一套经过严肃工程实践验证的 AI Agent 架构。它不是一个实验性 Demo，而是一个每天被大量开发者使用的生产级产品。

对 AI 产品经理来说，这套架构最值得学习的不是某个具体技术（双缓冲渲染、MCP 协议这些细节交给工程团队就好），而是它背后的设计理念：安全是架构的一部分而不是补丁，性能优化服务于用户体验而不是技术指标，可扩展性要在第一天而不是第 N 天设计。

这些理念听起来像正确的废话，但 Claude Code 用 13000 行的 QueryEngine、多层权限决策模型、自研渲染引擎证明了：把这些理念落地到每一行代码里，产品就是不一样。

题图来自Unsplash，基于CC0协议

384 次浏览

👍 6 次

相关文章

- [基于图卷积网络的图深度学习](#)
- [自动驾驶中的3D目标检测](#)
- [工业机器人控制系统架构介绍](#)
- [项目实战：如何构建知识图谱](#)



企业网络安全防护体系

5月20-21日 北京+直播